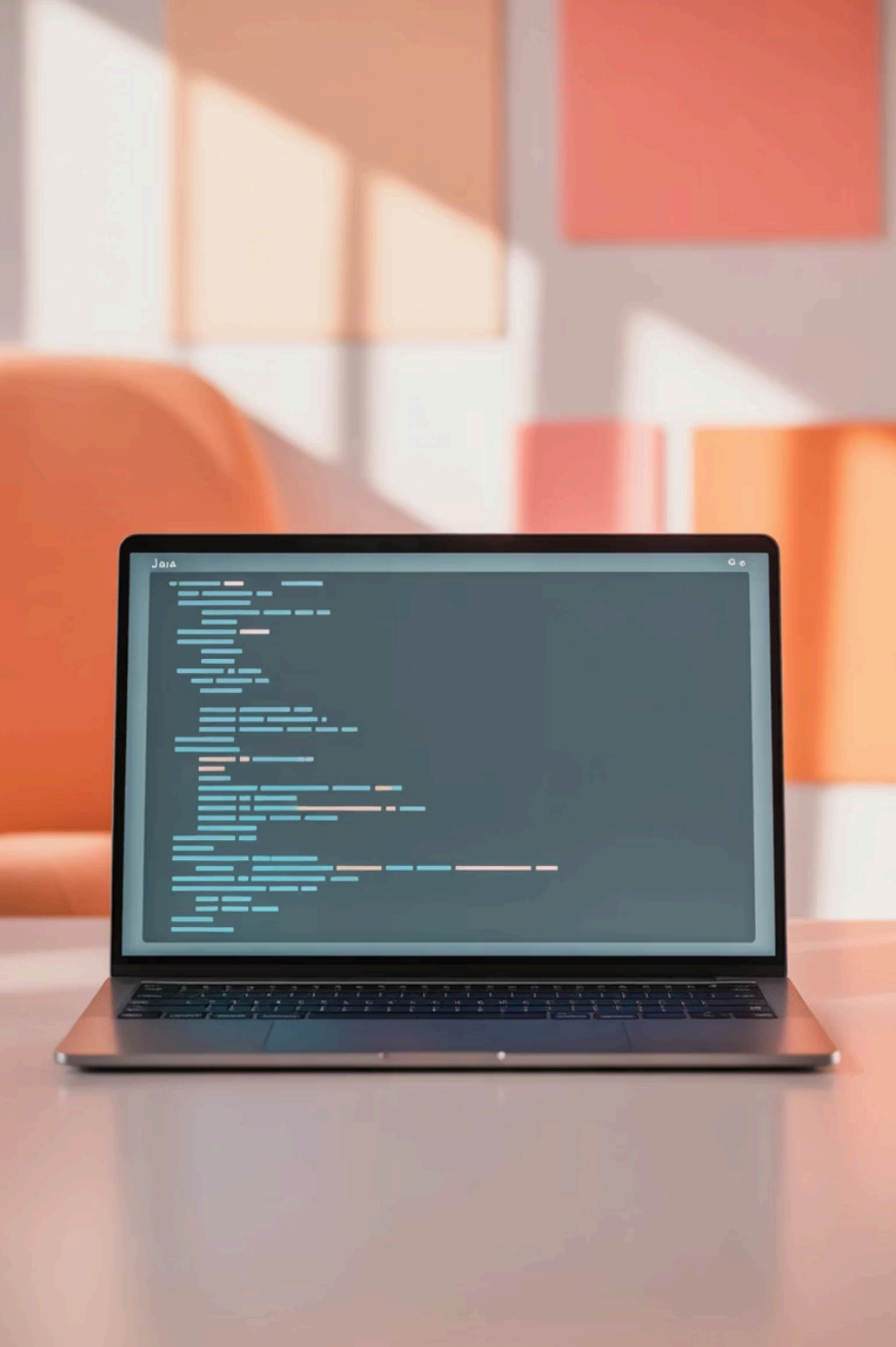




Lombok en Java: Reduce el código boilerplate con anotaciones inteligentes

Presentado por: Juan Cruz Robledo

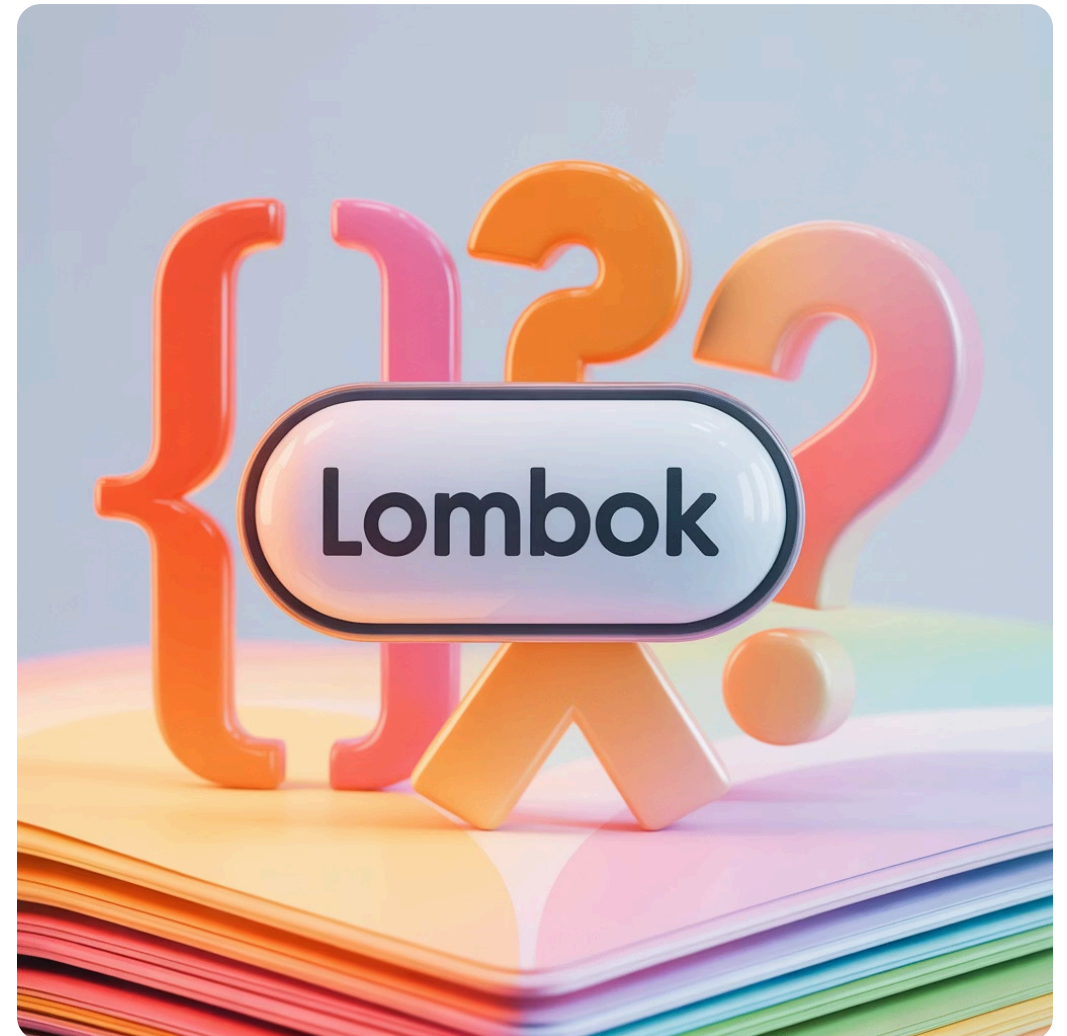
Mayo 2024

¿Qué es Lombok?

Lombok es una biblioteca de Java que:

- Reduce drásticamente el código repetitivo (boilerplate) mediante anotaciones
- Genera automáticamente código en tiempo de compilación
- Se integra perfectamente con el ecosistema de Java

Es especialmente popular en proyectos modernos como aplicaciones Spring Boot, microservicios y APIs REST donde la legibilidad y mantenibilidad son cruciales.



¿Por qué usar Lombok?

Ahorro de tiempo

Elimina la necesidad de escribir código repetitivo como getters, setters, constructores y métodos toString()

Mayor legibilidad

Reduce drásticamente el tamaño de las clases, permitiendo enfocarse en la lógica del negocio

Menos errores

Al generar código automáticamente, se reducen los errores humanos en métodos estándar

Un desarrollador promedio puede reducir hasta un 40% de líneas de código en sus clases de modelo utilizando Lombok.

Cómo agregar Lombok al proyecto

Maven

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.30</version>
  <scope>provided</scope>
</dependency>
```

Gradle

```
compileOnly 'org.projectlombok:lombok:1.18.30'
annotationProcessor 'org.projectlombok:lombok:1.18.30'
```

1

Configuración del IDE

Instala el plugin de Lombok en tu IDE (Eclipse o IntelliJ IDEA) para habilitar el soporte completo

2

Verificación

Comprueba que tu proyecto compile correctamente después de agregar Lombok

Anotaciones básicas de Lombok



@Getter / @Setter

Genera métodos getter y setter para todos los campos o específicos

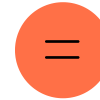
```
@Getter @Setter
private String nombre;
// Genera:
// public String getNombre() {...}
// public void setNombre(String
nombre) {...}
```



@ToString

Genera un método toString() personalizable

```
@ToString(exclude = "password")
public class Usuario {...}
// Genera un toString() sin incluir el
campo password
```



@EqualsAndHashCode

Genera los métodos equals() y hashCode() basados en los campos

```
@EqualsAndHashCode(of = {"id"})
public class Producto {...}
// Compara solo por el campo id
```

Anotaciones avanzadas de Lombok

@Builder

Genera un patrón builder completo para tu clase, permitiendo crear instancias de forma fluida

```
Usuario usuario = Usuario.builder()
    .nombre("Juan")
    .email("juan@ejemplo.com")
    .build();
```

@Data

Combinación de @Getter, @Setter, @ToString, @EqualsAndHashCode y @RequiredArgsConstructor

@Value

Similar a @Data pero para clases inmutables (todos los campos finales)

@Slf4j

Genera un logger estático compatible con SLF4J para facilitar el registro de eventos

```
@Slf4j
public class MiServicio {
    public void metodo() {
        log.info("Operación completada");
    }
}
```



Consideraciones y advertencias



El código no es visible

El código generado por Lombok no aparece en los archivos fuente, sino en los archivos .class después de la compilación



No es magia

Es crucial entender qué código está generando Lombok bajo el capó para evitar comportamientos inesperados



Problemas de IDE

Sin el plugin adecuado, los IDEs pueden mostrar errores falsos al no reconocer los métodos generados



Compatibilidad

Algunas herramientas de análisis de código pueden tener problemas con Lombok si no están configuradas correctamente

Recomendación: Revisa la documentación oficial de Lombok para comprender todas las implicaciones de cada anotación.

Buenas prácticas con Lombok

Especificidad sobre conveniencia

Prefiere anotaciones específicas (@Getter, @ToString) sobre anotaciones combinadas (@Data) cuando necesites control granular

Separación de responsabilidades

Evita mezclar lógica compleja con Lombok, especialmente en entidades JPA con relaciones complejas

Uso adecuado de @Data

Usa @Data solo en clases que representen estructuras de datos simples (DTOs, modelos básicos)

Documentación

Incluye comentarios que indiquen qué anotaciones estás usando, especialmente para otros desarrolladores menos familiarizados con Lombok

Ejemplo completo: Antes y después

Sin Lombok (70+ líneas)

```
public class Usuario {
    private Long id;
    private String nombre;
    private String email;
    private String password;
    private LocalDate fechaRegistro;

    public Usuario() {}

    public Usuario(Long id, String nombre,
        String email, String password) {
        this.id = id;
        this.nombre = nombre;
        this.email = email;
        this.password = password;
        this.fechaRegistro = LocalDate.now();
    }

    // Getters y setters (20+ líneas)
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNombre() { return nombre; }
    // ... más getters y setters ...

    // equals, hashCode, toString (15+ líneas)
    @Override
    public String toString() {
        return "Usuario{" +
            "id=" + id +
            ", nombre=" + nombre + "\" +
            // ... más campos ...
        }
    }
    // ... más código ...
}
```

Con Lombok (10 líneas)

```
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Usuario {
    private Long id;
    private String nombre;
    private String email;
    private String password;
    private LocalDate fechaRegistro = LocalDate.now();
}
```



Resumen: Potencia tu desarrollo con Lombok

Código más limpio

Elimina el código repetitivo para mejorar la legibilidad y el mantenimiento

Mayor productividad

Reduce el tiempo de desarrollo y permite enfocarse en la lógica del negocio

Uso consciente

Aprovecha el poder de Lombok entendiendo sus limitaciones y mejores prácticas

Recuerda: Lombok no es un reemplazo para entender Java, sino una herramienta para hacerte más productivo cuando ya comprendes los conceptos fundamentales.

✓ ¡Gracias por tu atención! ¿Preguntas?

Recursos adicionales: projectlombok.org/features/



¡Gracias!

Esperamos que Lombok impulse tu productividad y te ayude a escribir código más limpio y eficiente.

¡Hasta la próxima!